

Phụ lục A — Python cho chuỗi thời gian

Phụ lục này là **bản tra cứu**, không phải bài học. Buổi 3 dạy lại phần thời gian một cách đầy đủ; các buổi khác trở về đây khi cần nhắc một cú pháp. Mọi đoạn code dưới đây **đã chạy** với Python 3.12, pandas 3.0.5, numpy 2.5.3, polars 1.44.2, DuckDB 1.5.5 (ngày 2026-09-17), và output in đúng như chép lại. Riêng mục "pandas 2 và pandas 3" chạy thêm với pandas 2.3.3.

Vì sao có hai bản pandas trong khoá? pandas 3.0 phát hành tháng 1/2026. Tới 2026-09-17, statsforecast, utilsforecast, mlforecast và autogluon.timeseries bản mới nhất vẫn yêu cầu `pandas<3`. Vì vậy buổi 1–13 dùng pandas 3.0.5, còn buổi nào dùng các thư viện đó thì môi trường được chốt ở pandas 2.3.3. Mỗi buổi có môi trường riêng nên hai bản không đụng nhau — nhưng bạn cần biết chỗ khác nhau (mục 3).

Mục lục

- 1. Môi trường của một buổi
- 2. NumPy tối thiểu
- 3. pandas 2 và pandas 3 — những chỗ khác nhau
- 4. Thời gian trong pandas
- 5. Múi giờ và DST
- 6. Định dạng dài và định dạng rộng
- 7. polars
- 8. DuckDB
- 9. Bảng đối chiếu nhanh
- 10. Bẫy thường gặp
- Nguồn

1. Môi trường của một buổi

Mỗi buổi có môi trường riêng trong `lab/00-nen/.venv`, dựng bằng `make up` (xem `MOI-TRUONG.md`). Chạy lệnh Python trong môi trường đó từ thư mục `lab/`:

```
env -u VIRTUAL_ENV uv run --no-sync --project 00-nen python ../code/vi_du.py
```

Code của khoá viết dạng **percent** của jupyter: `# %%` mở một ô code, `# %% [markdown]` mở một ô chữ. Tập vẫn là Python hợp lệ nên chạy được như script và diff được trong git.

```
# %% [markdown]
# # Tiêu đề notebook

# %%
import pandas as pd
```

Thư viện trợ giúp của buổi là gói `tv` (cài sẵn trong môi trường). `tv.THU_MUC_DU_LIEU` là thư mục dữ liệu đã kiểm sha256, dùng thay cho đường dẫn gõ tay.

2. NumPy tối thiểu

Sinh số ngẫu nhiên có seed — dùng `Generator`, không dùng `np.random.seed` toàn cục:

```
rng = np.random.default_rng(42)
rng.normal(size=3).round(4)      # array([ 0.3047, -1.04 ,  0.7505])
```

Quantile thực nghiệm. `np.quantile` có 9 cách tính theo phân loại của Hyndman & Fan (1996); mặc định `method="linear"` là loại 7. Cùng dữ liệu, khác phương pháp cho khác kết quả:

```
np.quantile([1, 2, 3, 4], 0.25)          # 1.75   (loại 7, mặc định)
np.quantile([1, 2, 3, 4], 0.25, method="median_unbiased") # 1.4167 (loại 8)
np.quantile([1, 2, 3, 4], 0.25, method="inverted_cdf")   # 1     (loại 1)
```

Khi so kết quả với R hay thư viện khác, kiểm cả phương pháp tính quantile trước khi nghi code sai.

3. pandas 2 và pandas 3 — những chỗ khác nhau

Chủ đề	pandas 2.3.3	pandas 3.0.5
Gán qua hai lần <code>[]</code> : <code>df["a"][mask] = v</code>	CÓ tác dụng , kèm <code>FutureWarning</code>	không tác dụng , báo <code>ChainedAssignmentError</code>
Cột chuỗi	<code>object</code>	<code>str</code>
<code>pd.to_datetime</code> từ chuỗi	<code>datetime64[ns]</code>	<code>datetime64[us]</code> (microsecond)
Bí danh tần suất <code>H</code> , <code>T</code> , <code>M</code>	chạy, kèm cảnh báo <code>deprecated</code>	<code>ValueError: Invalid frequency</code>
<code>fillna(method="ffill")</code>	chạy, kèm <code>FutureWarning</code>	<code>TypeError</code> — dùng <code>.ffill()</code>
Đối tượng múi giờ	<code>pytz</code>	<code>zoneinfo</code> (<code>pytz</code> không còn là phụ thuộc)
<code>groupby(..., observed=)</code> mặc định	<code>False</code>	<code>True</code>

Viết code chạy được trên cả hai bản: luôn gán bằng `df.loc[mask, "a"] = v`; dùng bí danh mới (`h`, `min`, `s`, `ME`, `QE`, `YE`); dùng `.ffill()` / `.bfill()`; không giả định đơn vị `ns` khi đổi `datetime` sang số nguyên.

Ví dụ chạy trên pandas 3.0.5:

```
df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
df["a"][df["b"] > 4] = 100      # ChainedAssignmentError, df KHÔNG đổi
df["a"].tolist()                # [1, 2, 3]
df.loc[df["b"] > 4, "a"] = 100  # cách đúng
df["a"].tolist()                # [1, 100, 100]
```

Ghi chú phát hành pandas 3.0 giải thích: với Copy-on-Write, mọi tập con trả về "always behaves as a copy of the original", nên gán chuỗi "will stop working".

4. Thời gian trong pandas

Bí danh tần suất

Bí danh	Nghĩa	Nhãn đặt ở
s, min, h	giây, phút, giờ	đầu khoảng
D	ngày lịch	đầu ngày
W-MON	tuần kết thúc thứ Hai	cuối tuần
MS / ME	đầu tháng / cuối tháng	ngày 1 / ngày cuối tháng
QE, YE	cuối quý, cuối năm	cuối kỳ

resample : nhãn và biên của khoảng

Giá trị gán nhãn 10:00 là của 9:00–10:00 hay 10:00–11:00? pandas quy định: `label` và `closed` mặc định là `'left'` cho mọi tần suất, **trừ** `ME`, `YE`, `QE`, `BME`, `BA`, `BQE` và `W` mặc định `'right'`.

```
y = pd.Series(range(6), index=pd.date_range("2024-01-01 00:00", periods=6, freq="h"))
y.resample("2h").sum()
# 00:00 → 1    (0 + 1: giờ 00 và 01)
# 02:00 → 5    (2 + 3)
# 04:00 → 9    (4 + 5)
```

Tổng hay trung bình phụ thuộc loại biến: **lưu lượng** (số chuyển, kWh) cộng lại; **trạng thái** (nhiệt độ, giá) lấy trung bình hoặc giá trị cuối.

Hướng thời gian của các phép cửa sổ

Hàm	Dùng dữ liệu nào tại thời điểm t	Dùng làm feature được không
<code>shift(1)</code>	y_{t-1}	được
<code>diff()</code>	$y_t - y_{t-1}$	chỉ khi y_t đã biết lúc dự báo
<code>rolling(3).mean()</code>	y_{t-2}, y_{t-1}, y_t	phải <code>shift</code> thêm nếu y_t là mục tiêu
<code>rolling(3, center=True).mean()</code>	y_{t-1}, y_t, y_{t+1}	không — dùng tương lai
<code>ewm(alpha).mean()</code>	mọi quá khứ tới t	phải <code>shift</code> thêm nếu y_t là mục tiêu

```
x = pd.Series([0.0, 1, 2, 3, 4, 5], index=pd.date_range("2024-01-01", periods=6, freq="h"))
x.shift(1).tolist()           # [nan, 0.0, 1.0, 2.0, 3.0, 4.0]
x.diff().tolist()           # [nan, 1.0, 1.0, 1.0, 1.0, 1.0]
x.rolling(3).mean().tolist() # [nan, nan, 1.0, 2.0, 3.0, 4.0]
x.rolling(3, center=True).mean().tolist() # [nan, 1.0, 2.0, 3.0, 4.0, nan] ← giá trị ở t=1 dùng x[2]
```

Cửa sổ theo **khoảng thời gian** (`"2h"`) có số điểm thay đổi, mặc định `closed='right'`:

```
x.rolling("2h").sum().tolist()           # [0.0, 1.0, 3.0, 5.0, 7.0, 9.0]    (t-2h, t]
x.rolling("2h", closed="both").sum().tolist() # [0.0, 1.0, 3.0, 6.0, 9.0, 12.0]   [t-2h, t]
```

`ewm` có hai dạng: `adjust=True` (mặc định) chuẩn hoá trọng số trên số điểm đã có; `adjust=False` là công thức đệ quy $s_t = \alpha y_t + (1 - \alpha)s_{t-1}$:

```
pd.Series([1.0, 2, 3]).ewm(alpha=0.5).mean().round(3).tolist()      # [1.0, 1.667, 2.429]
pd.Series([1.0, 2, 3]).ewm(alpha=0.5, adjust=False).mean().tolist() # [1.0, 1.5, 2.25]
```

`merge_asof` — ghép với giá trị gần nhất trong quá khứ

Dùng để ghép hai nguồn khác tần suất mà không lấy thông tin tương lai. Cả hai bảng phải **sắp tăng dần** theo khoá; mặc định `direction='backward'` chọn dòng bên phải có khoá $\leq$ khoá bên trái.

```
traai = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:05"])})
phai = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:03"]), "v": [1, 2]})
pd.merge_asof(traai, phai, on="t")["v"].tolist()      # [1, 2]
pd.merge_asof(traai, phai, on="t", allow_exact_matches=False)["v"].tolist() # [nan, 2.0]
```

`allow_exact_matches=False` dùng khi giá trị tại đúng thời điểm t chưa công bố kịp lúc dự báo.

5. Múi giờ và DST

Quy tắc của khoá: **lưu trữ theo UTC, chỉ đổi sang giờ địa phương khi hiển thị hoặc khi tính đặc trưng lịch** (giờ trong ngày, ngày trong tuần).

- **Naive** (không múi giờ) vs **aware** (có múi giờ): `tz_localize` gán múi giờ cho dữ liệu naive, `tz_convert` đổi giữa các múi giờ.
- **Việt Nam** (`Asia/Ho_Chi_Minh`) hiện là UTC+7 không có giờ mùa hè — nhưng theo cơ sở dữ liệu múi giờ IANA, điều này chỉ đúng **từ 13/6/1975**; trước đó có các độ lệch khác:

```
pd.Timestamp("2024-07-01 12:00", tz="Asia/Ho_Chi_Minh").tz_convert("UTC")      # 2024-07-01
05:00:00+00:00
zoneinfo.ZoneInfo("Asia/Ho_Chi_Minh").utcoffset(datetime(1970, 1, 1))          # 8:00:00
```

- **Nơi có DST** (Mỹ, châu Âu): ngày chuyển sang giờ mùa hè **mất** một giờ, ngày chuyển về **lặp** một giờ. Dữ liệu điện EIA, taxi NYC đều gặp.

```
pd.date_range("2024-03-10 00:00", periods=4, freq="h", tz="America/New_York")
# 00:00-05:00, 01:00-05:00, 03:00-04:00, 04:00-04:00 ← không có 02:00
```

Gắn múi giờ cho giờ nằm trong vùng lặp hoặc vùng mất sẽ lỗi, trừ khi nói rõ cách xử lý:

```
t = pd.Series(pd.to_datetime(["2024-11-03 01:30"]))
t.dt.tz_localize("America/New_York")
# ValueError: Cannot infer dst time from 2024-11-03 01:30:00, try using the 'ambiguous' argument
t.dt.tz_localize("America/New_York", ambiguous=np.array([True]))      # 01:30-04:00 (bản giờ mùa hè)
pd.Series(pd.to_datetime(["2024-03-10 02:30"])).dt.tz_localize(
    "America/New_York", nonexistent="shift_forward")                  # 03:00-04:00
```

Tham số `ambiguous` nhận `'infer'`, `'NaT'`, `'raise'` hoặc mảng bool (`True` = giờ mùa hè); `nonexistent` nhận `'shift_forward'`, `'shift_backward'`, `'NaT'`, một `timedelta` hoặc `'raise'`.

Trên máy không có cơ sở dữ liệu múi giờ IANA (đáng chú ý là Windows), tài liệu Python khuyến nghị khai phụ thuộc `tzdata` — môi trường của mọi buổi đã có sẵn.

6. Định dạng dài và định dạng rộng

Hệ thư viện Nixtla (statsforecast, mlforecast, neuralforecast...) dùng **định dạng dài** ba cột `unique_id`, `ds`, `y`: mỗi dòng là một quan sát của một chuỗi tại một thời điểm.

```
rong = pd.DataFrame({"ds": pd.date_range("2024-01-01", periods=2, freq="D"),
                    "HN": [10, 11], "HCM": [20, 21]})
dai = rong.melt(id_vars="ds", var_name="unique_id", value_name="y")[["unique_id", "ds", "y"]]
# unique_id      ds    y
#          HN 2024-01-01  10
#          HN 2024-01-02  11
#          HCM 2024-01-01  20
#          HCM 2024-01-02  21
dai.pivot(index="ds", columns="unique_id", values="y")    # quay lại dạng rộng
```

Dạng dài chứa được chuỗi dài ngắn khác nhau và thêm cột đặc trưng dễ; dạng rộng tiện để vẽ nhiều chuỗi và tính tương quan giữa các chuỗi.

7. polars

polars xử lý theo cột, có chế độ lazy (`scan_parquet`) tối ưu cả truy vấn trước khi chạy — hợp với dữ liệu vài chục triệu dòng.

```
d = pl.DataFrame({"t": pd.date_range("2024-01-01", periods=6, freq="h"), "v": [0, 1, 2, 3, 4, 5]})
d.group_by_dynamic("t", every="2h").agg(pl.col("v").sum())    # t: 00:00, 02:00, 04:00; v: 1, 5, 9
```

`group_by_dynamic` tương đương `resample`: mặc định `closed='left'`, `label='left'`, cột thời gian phải sắp tăng dần. `join_asof` tương đương `merge_asof`, mặc định `strategy='backward'`, hai bảng phải sắp theo khoá:

```
q.join_asof(r, on="t")["v"].to_list()    # [1, 2] — cùng dữ liệu ví dụ merge_asof ở mục 4
```

Múi giờ: `dt.replace_time_zone("UTC")` gán múi giờ, `dt.convert_time_zone("Asia/Ho_Chi_Minh")` đổi:

```
d.with_columns(pl.col("t").dt.replace_time_zone("UTC")
               .dt.convert_time_zone("Asia/Ho_Chi_Minh"))["t"][0]
# 2024-01-01 07:00:00+07:00
```

Tài liệu polars lưu ý `replace_time_zone` "will also modify the underlying timestamp" — nó không phải phép đổi múi giờ; nhầm hai hàm là lệch giờ âm thầm.

8. DuckDB

DuckDB chạy SQL thẳng trên tệp Parquet/CSV, không cần máy chủ.

```
-- ghép giá trị gần nhất trong quá khứ
SELECT trai.t, phai.v FROM trai ASOF JOIN phai ON trai.t >= phai.t;
-- (10:00, 1), (10:05, 2)

SELECT time_bucket(INTERVAL '15 minutes', TIMESTAMP '2024-01-01 10:07'); -- 10:00
SELECT date_trunc('hour', TIMESTAMP '2024-01-01 10:07'); -- 10:00
SELECT * FROM 'du-lieu.parquet'; -- đọc tệp trực tiếp
```

TIMESTAMPZ của DuckDB **không lưu múi giờ**: nó lưu số microsecond kể từ epoch, còn hiển thị và phép chia khoảng theo cài đặt **TimeZone** (mặc định là múi giờ của máy):

```
SET TimeZone = 'Asia/Ho_Chi_Minh';
SELECT (TIMESTAMPZ '2024-01-01 00:00:00+00')::VARCHAR; -- '2024-01-01 07:00:00+07'
```

Hệ quả: cùng một truy vấn `date_trunc('day', col_timestampz)` cho kết quả khác nhau trên máy đặt múi giờ khác nhau — luôn `SET TimeZone` ở đầu script.

Với pandas 3, lấy cột **TIMESTAMPZ** từ DuckDB về Python cần cài **pytz** (pandas 3 không kéo theo pytz nữa); thiếu sẽ báo "Required module 'pytz' failed to import".

9. Bảng đối chiếu nhanh

Việc	pandas	polars	DuckDB
gộp theo khoảng thời gian	<code>resample("h")</code>	<code>group_by_dynamic("t", every="1h")</code>	<code>time_bucket(INTERVAL '1 hour', t)</code>
ghép quá khứ gần nhất	<code>merge_asof(..., direction="backward")</code>	<code>join_asof(..., strategy="backward")</code>	<code>ASOF JOIN ... ON a.t >= b.t</code>
trễ 1 bước	<code>shift(1)</code>	<code>pl.col("y").shift(1)</code>	<code>lag(y) OVER (ORDER BY t)</code>
gắn / đổi múi giờ	<code>tz_localize / tz_convert</code>	<code>dt.replace_time_zone / dt.convert_time_zone</code>	<code>SET TimeZone + TIMESTAMPZ</code>

10. Bẫy thường gặp

Bẫy	Triệu chứng	Sửa
Gán <code>df["a"][mask] = v</code> trên pandas 3	cột không đổi, có <code>ChainedAssignmentError</code>	<code>df.loc[mask, "a"] = v</code>
Bí danh cũ <code>H, M, T</code>	<code>ValueError: Invalid frequency</code> trên pandas 3	<code>h, ME, min</code>
Giả định <code>datetime</code> là <code>ns</code>	<code>astype("int64")</code> ra số nhỏ hơn 1.000 lần trên pandas 3	đổi đơn vị tường minh <code>.dt.as_unit("ns")</code>
<code>rolling(center=True)</code> làm feature	backtest đẹp giả tạo	cửa sổ lùi + <code>shift</code>
<code>merge_asof</code> với bảng chưa sắp	<code>ValueError: left keys must be sorted</code>	<code>sort_values</code> theo khoá trước

Bẫy	Triệu chứng	Sửa
Dữ liệu naive trộn nhiều múi giờ	"mưa gây ra tăng chuyển trước khi mưa"	chuẩn hoá về UTC ngay khi đọc
Nghĩ Việt Nam luôn là UTC+7	lịch giờ dữ liệu trước 1975	dùng <code>zoneinfo</code> , không cộng tay 7 giờ
<code>replace_time_zone</code> thay cho <code>convert_time_zone</code> (polars)	lịch giờ âm thầm	đọc kỹ: replace đổi cách hiểu, convert đổi giờ
DuckDB <code>date_trunc</code> trên <code>TIMESTAMPTZ</code>	kết quả khác nhau giữa các máy	<code>SET TimeZone</code> đầu script

Nguồn

Truy cập ngày 2026-09-17. Nhật ký research đầy đủ: `tools/NGHIEN-CUU.md`.

- pandas 3.0.0 — What's new: <https://pandas.pydata.org/docs/whatsnew/v3.0.0.html>
- pandas `DataFrame.resample`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.resample.html>
- pandas `Series.tz_localize`: https://pandas.pydata.org/docs/reference/api/pandas.Series.tz_localize.html
- pandas `merge_asof`: https://pandas.pydata.org/docs/reference/api/pandas.merge_asof.html
- pandas `DataFrame.rolling`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html>
- NumPy `quantile`: <https://numpy.org/doc/stable/reference/generated/numpy.quantile.html>
- Hyndman, R.J. & Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician* 50(4), 361–365.
- Python `zoneinfo`: <https://docs.python.org/3/library/zoneinfo.html>
- IANA tz database, tệp `asia`: <https://data.iana.org/time-zones/tzdb/asia>
- polars `join_asof`: https://docs.pola.rs/api/python/stable/reference/dataframe/api/polars.DataFrame.join_asof.html
- polars `group_by_dynamic`, `dt.replace_time_zone`: <https://docs.pola.rs/api/python/stable/reference/>
- DuckDB ASOF JOIN: https://duckdb.org/docs/current/guides/sql_features/asof_join.html
- DuckDB timestamp functions: <https://duckdb.org/docs/current/sql/functions/timestamp.html>
- DuckDB `TIMESTAMP` / `TIMESTAMPTZ`: https://duckdb.org/docs/current/sql/data_types/timestamp.html
- jupyter text percent format: <https://jupyter.org/formats/scripts/>